stat 432 hw8

Lu Gong

HW8

question 1

a.

For the given rule $f(x) = 0.8x_1 + 0.6x_2$, the normal vector has length 1. Therefore, the distance to the decision boundary is

$$d = \frac{|f(x)|}{\sqrt{0.8^2 + 0.6^2}} = |f(x)|.$$

The functional margin is $m = yf(x)$. An observation is inside its class margin when $m < 1$, on the margin when $m = 1$, and beyond the margin when $m > 1$. The calculation uses a tolerance of 10^{-8} when checking whether the margin equals 1.

```
dat <- read.csv("data/margin-geometry-20.csv")
stopifnot(
  nrow(dat) == 20,
  identical(dat$observation, 1:20),
  !anyNA(dat),
  all(dat$y %in% c(-1, 1))
)

score <- 0.8 * dat$x1 + 0.6 * dat$x2
distance <- abs(score) / sqrt(0.8^2 + 0.6^2)
margin <- dat$y * score
tol <- 1e-8

position <- ifelse(
  abs(margin - 1) < tol,
  "On",
  ifelse(margin < 1, "Inside", "Beyond")
)

result <- data.frame(
  Observation = dat$observation,
  Score = score,
  Distance = distance,
  Functional_margin = margin,
  Position = position
)

knitr::kable(
  result,
  digits = 2,
  col.names = c(
    "Observation", "Score", "Distance", "Functional margin", "Position"
  ),
  caption = "Distances and functional margins for all 20 observations."
)
```

Table 1: Distances and functional margins for all 20 observations.

Observation	Score	Distance	Functional margin	Position
1	1.00	1.00	1.00	On
2	1.35	1.35	1.35	Beyond
3	1.50	1.50	1.50	Beyond
4	1.70	1.70	1.70	Beyond
5	1.90	1.90	1.90	Beyond
6	2.20	2.20	2.20	Beyond
7	2.40	2.40	2.40	Beyond
8	0.64	0.64	0.64	Inside
9	0.20	0.20	0.20	Inside
10	-0.20	0.20	-0.20	Inside
11	-1.00	1.00	1.00	On
12	-1.30	1.30	1.30	Beyond
13	-1.55	1.55	1.55	Beyond
14	-1.75	1.75	1.75	Beyond
15	-1.95	1.95	1.95	Beyond
16	-2.15	2.15	2.15	Beyond
17	-2.45	2.45	2.45	Beyond
18	-0.64	0.64	0.64	Inside
19	-0.24	0.24	0.24	Inside
20	0.20	0.20	-0.20	Inside

b.

The solid line shows the decision boundary $f(x) = 0$, while the dashed lines show the class margins $f(x) = \pm 1$. Different colors and shapes are used for the two classes. The observations inside their class margin are circled and labeled, including the misclassified observations.

```
library(ggplot2)

dat$class <- factor(
  dat$y,
  levels = c(-1, 1),
  labels = c("-1", "+1")
)
inside <- dat[position == "Inside", ]

p <- ggplot(dat, aes(x = x1, y = x2)) +
  geom_abline(intercept = 0, slope = -0.8 / 0.6) +
  geom_abline(
    intercept = c(-1 / 0.6, 1 / 0.6),
    slope = -0.8 / 0.6,
    linetype = "dashed",
    colour = "grey50"
  ) +
  geom_point(aes(colour = class, shape = class), size = 3) +
  geom_point(
    data = inside,
    shape = 1, size = 5, colour = "black"
  ) +
  geom_text(
    data = inside,
    aes(label = observation),
    vjust = -1.2, size = 3
  ) +
  scale_colour_manual(values = c("#D55E00", "#0072B2")) +
  scale_shape_manual(values = c(17, 16)) +
  coord_fixed(
    ratio = 1,
    xlim = c(-3.2, 3.2),
    ylim = c(-3, 3),
```

```
    expand = FALSE
  ) +
  labs(
    x = "x1", y = "x2",
    colour = "True class", shape = "True class",
    title = "Decision boundary and class margins",
    caption = paste(
      "Solid: f(x) = 0; dashed: f(x) = +/-1.",
      "Circled and numbered points are inside their class margin.",
      sep = "\n"
    )
  ) +
  theme_minimal()

print(p)
```

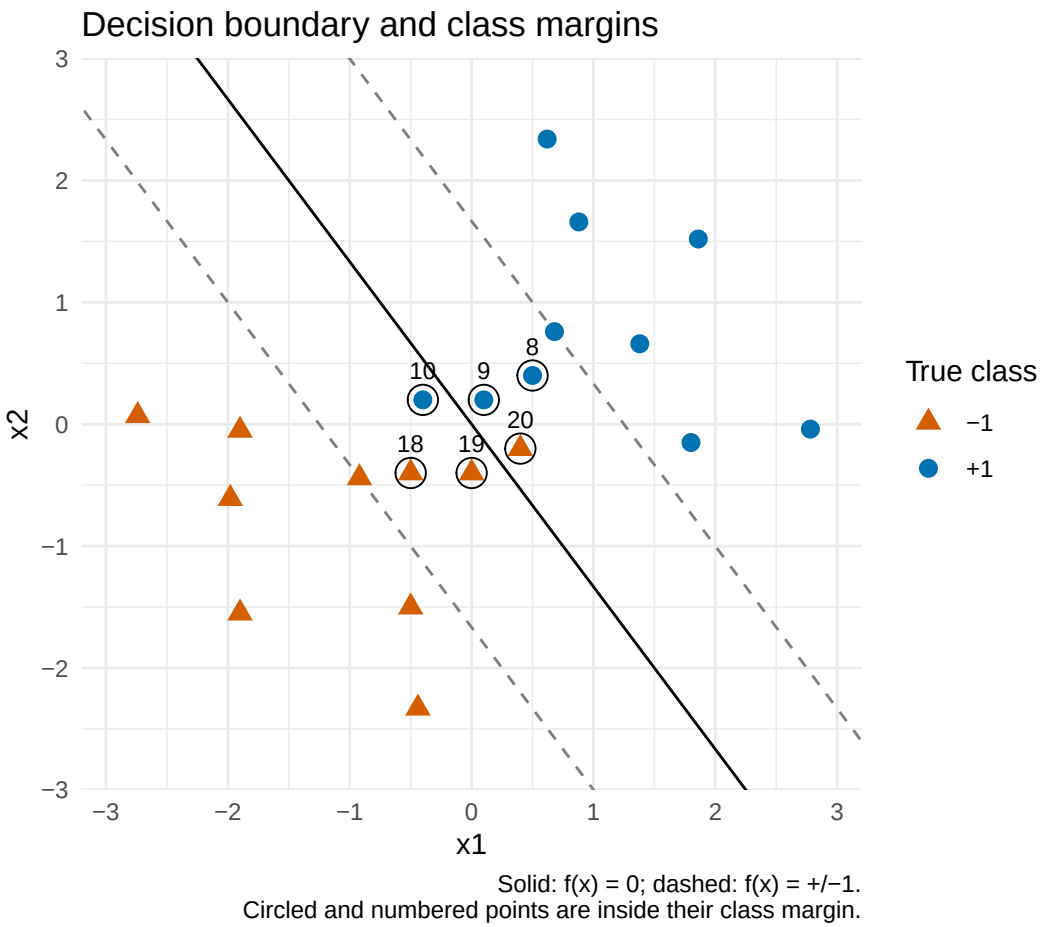


Figure 1

c.

```
counts <- data.frame(
  Position = c("Inside", "On", "Beyond"),
  Count = c(
    sum(position == "Inside"),
    sum(position == "On"),
```

```
    sum(position == "Beyond")
  )
)
misclassified <- dat$observation[margin < -tol]

knitr::kable(counts)
```

Position	Count
Inside	6
On	2
Beyond	12

There are 6 observations inside the class margin, 2 on the margin, and 12 beyond it. Observations 10 and 20 are misclassified because both have a negative functional margin of -0.20. However, being inside the margin does not necessarily mean an observation is misclassified. It can still be correctly classified when its functional margin is positive but less than 1.

Data: `margin-geometry-20.csv` from HW08 Q1.

question 2

a.

Using the four predictors $\mathbf{x}_i = (x_{i1}, x_{i2}, x_{i3}, x_{i4})^\mathsf{T}$, the score for observation i is

$$f(\mathbf{x}_i) = \beta_0 + \mathbf{x}_i^\mathsf{T}\beta = -1 + x_{i1} + 2x_{i2} - 2x_{i3} + 4x_{i4},$$

The intercept is $\beta_0 = -1$, and $\beta = (1, 2, -2, 4)^\mathsf{T}$ gives the direction perpendicular to the boundary. Its length is 5, so dividing by 5 gives the unit direction:

$$\|\beta\|_2 = \sqrt{1^2 + 2^2 + (-2)^2 + 4^2} = 5, \qquad \mathbf{u} = \frac{\beta}{5} = (0.2, 0.4, -0.4, 0.8)^\mathsf{T}.$$

The coordinate after projecting each observation onto this direction is

$$z_i = \mathbf{x}_i^\mathsf{T}\mathbf{u} = \frac{x_{i1} + 2x_{i2} - 2x_{i3} + 4x_{i4}}{5}.$$

Here z_i is the coordinate on the projection axis, while $z_i\mathbf{u}$ would be the projection vector. Using the unit direction keeps distances on this axis in the original units. No centering or scaling of the predictors was used.

The boundary satisfies $\beta_0 + \|\beta\|_2 z_0 = 0$. Therefore, its coordinate is

$$z_0 = -\frac{\beta_0}{\|\beta\|_2} = \frac{1}{5} = 0.2.$$

The perpendicular distance is $d_i = |z_i - z_0| = |f(\mathbf{x}_i)|/5$. Since all observations are correctly classified, the geometric margin is the smallest of these distances:

$$M = \min_i d_i = \min_i |z_i - 0.2|.$$

```r
dat4 <- read.csv("data/margin-geometry-4d.csv")
stopifnot(
  nrow(dat4) == 50,
  identical(dat4$observation, 1:50),
  !anyNA(dat4),
  all(dat4$y %in% c(-1, 1))
)

X4 <- as.matrix(dat4[, c("x1", "x2", "x3", "x4")])
beta <- c(1, 2, -2, 4)
beta0 <- -1
beta_length <- sqrt(sum(beta^2))
unit_direction <- beta / beta_length

projection4 <- as.vector(X4 %*% unit_direction)
boundary4 <- -beta0 / beta_length
score4 <- beta0 + as.vector(X4 %*% beta)
functional4 <- dat4$y * score4
distance4 <- abs(projection4 - boundary4)

M <- min(distance4)
tol4 <- 1e-8
nearest4 <- abs(distance4 - M) < tol4

stopifnot(
  max(abs(distance4 - abs(score4) / beta_length)) < tol4,
  all(functional4 > 0)
)

nearest_table <- data.frame(
  Observation = dat4$observation[nearest4],
  Class = dat4$y[nearest4],
  Projection = projection4[nearest4],
  Score = score4[nearest4],
  Functional_margin = functional4[nearest4],
  Distance = distance4[nearest4]
)
knitr::kable(
  nearest_table,
  digits = 2,
  caption = "All observations attaining the smallest distance."
)
```

Table 2: All observations attaining the smallest distance.

Observation	Class	Projection	Score	Functional_margin	Distance
4	1	1.2	5	5	1
15	-1	-0.8	-5	5	1
33	1	1.2	5	5	1
36	-1	-0.8	-5	5	1

The results show that observations 4, 15, 33, 36 are closest to the boundary. The nearest negative-class observations project to -0.8, while the nearest positive-class observations project to 1.2. Therefore,

$$M = \min\{|-0.8 - 0.2|,\ |1.2 - 0.2|\} = 1.$$

The geometric margin is 1, which is the distance from the boundary to the nearest observation on either side. The full width between the two margins is 2. All 50 observations have positive functional margins, so the given rule classifies them correctly.

```r
projection_plot <- data.frame(
  observation = dat4$observation,
  z = projection4,
  class = factor(dat4$y, levels = c(-1, 1), labels = c("-1", "+1"))
)

# Stack equal projections without changing any horizontal coordinate.
projection_plot$stack <- (
  ave(dat4$observation, round(projection4, 8), FUN = seq_along) - 1
) * 0.10

margin_arrows <- data.frame(
  start = c(boundary4 - M, boundary4),
  end = c(boundary4, boundary4 + M)
)

p4 <- ggplot(projection_plot, aes(x = z, y = stack)) +
  geom_hline(yintercept = 0, colour = "grey75") +
  geom_vline(xintercept = boundary4, linewidth = 0.6) +
  geom_vline(
    xintercept = c(boundary4 - M, boundary4 + M),
    linetype = "dashed", colour = "grey50"
  ) +
  geom_point(aes(colour = class, shape = class), size = 2.5) +
  geom_point(
    data = projection_plot[nearest4, ],
    shape = 1, size = 5, colour = "black"
  ) +
  geom_text(
    data = projection_plot[nearest4, ],
    aes(label = observation), vjust = -1.1, size = 3
  ) +
  geom_segment(
    data = margin_arrows,
    aes(x = start, xend = end, y = 0.34, yend = 0.34),
    inherit.aes = FALSE,
    arrow = grid::arrow(length = grid::unit(0.07, "inches"), ends = "both")
  ) +
  annotate(
    "text", x = c(boundary4 - M / 2, boundary4 + M / 2),
    y = 0.43, label = paste0("M = ", round(M, 2)), size = 3.5
  ) +
  annotate("text", x = boundary4 + 0.10, y = -0.10,
           label = "boundary: 0.2", hjust = 0, size = 3) +
  scale_colour_manual(values = c("#D55E00", "#0072B2")) +
  scale_shape_manual(values = c(17, 16)) +
  scale_y_continuous(breaks = NULL) +
  coord_cartesian(ylim = c(-0.14, 0.50)) +
  labs(
    x = "Projection coordinate z = x^T beta / ||beta||", y = NULL,
    colour = "True class", shape = "True class",
    title = "Four-dimensional observations projected onto one axis",
    caption = "Vertical stacking separates coincident points; only the horizontal axis represents distance."
  ) +
  theme_minimal() +
  theme(legend.position = "bottom", panel.grid = element_blank())

print(p4)
```

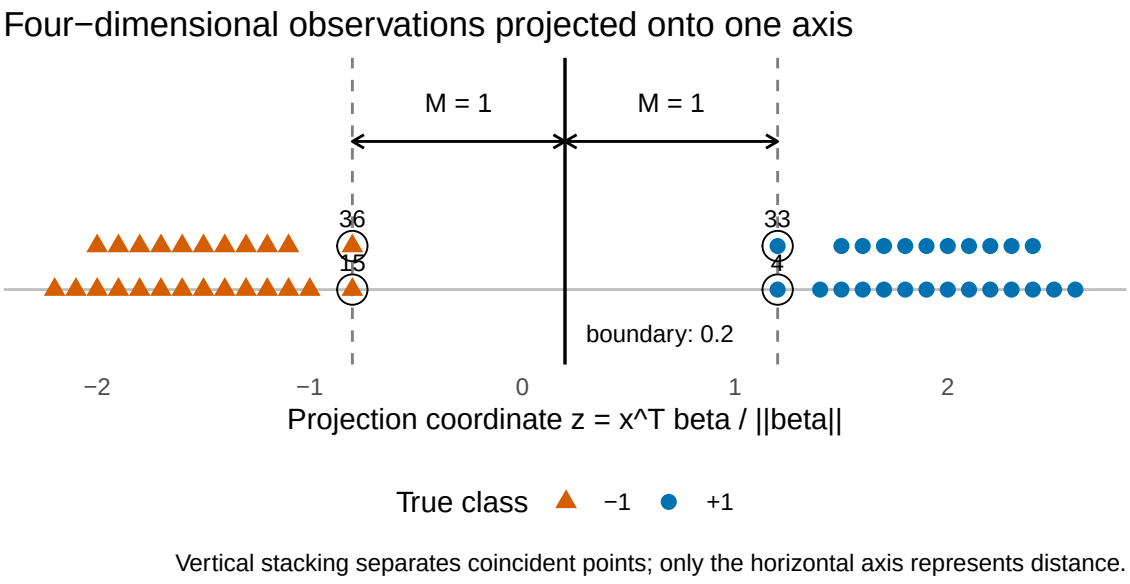


Figure 2

b.

The smallest functional margin is $m_{\min} = \min_i y_i f(\mathbf{x}_i)$, where $y_i \in \{-1, 1\}$ is the true class. All functional margins are positive in this dataset. Dividing both the intercept and the coefficients by $m_{\min}$ makes the smallest functional margin equal to 1:

$$\beta_0^* = \frac{\beta_0}{m_{\min}}, \qquad \beta^* = \frac{\beta}{m_{\min}}.$$

The rescaled rule is $f^*(\mathbf{x}) = f(\mathbf{x})/m_{\min}$, giving $\min_i y_i f^*(\mathbf{x}_i) = 1$. Since $m_{\min}$ is positive, the signs of the scores and the zero boundary do not change. Therefore, this gives the same decision boundary and predictions without fitting a new model.

```
min_functional4 <- min(functional4)
stopifnot(min_functional4 > 0)

beta0_svm <- beta0 / min_functional4
beta_svm <- beta / min_functional4
score4_svm <- beta0_svm + as.vector(X4 %*% beta_svm)
functional4_svm <- dat4$y * score4_svm
boundary4_svm <- -beta0_svm / sqrt(sum(beta_svm^2))
distance4_svm <- abs(score4_svm) / sqrt(sum(beta_svm^2))

stopifnot(
  abs(min(functional4_svm) - 1) < tol4,
  max(abs(score4_svm - score4 / min_functional4)) < tol4,
  all(sign(score4_svm) == sign(score4)),
  abs(boundary4_svm - boundary4) < tol4,
  max(abs(distance4_svm - distance4)) < tol4
)

knitr::kable(
  data.frame(
    Parameter = c("Intercept", "x1", "x2", "x3", "x4"),
    Original = c(beta0, beta),
    Normalized = c(beta0_svm, beta_svm)
  ),
  digits = 2
)
```

Parameter	Original	Normalized
Intercept	-1	-0.2
x1	1	0.2
x2	2	0.4
x3	-2	-0.4
x4	4	0.8

The smallest original functional margin is 5.00. Dividing by this value gives

$$\boxed{\beta_0^* = -0.2, \qquad \beta^* = (0.2, 0.4, -0.4, 0.8)^\mathsf{T}.}$$

The new minimum functional margin is 1.00, while the geometric margin remains 1.00. The new score is $f^*(\mathbf{x}_i) = z_i - 0.2$, so the boundary is still $z = 0.2$.

However, projection and SVM normalization use different quantities. Projection divides β by its length, while SVM normalization divides both β_0 and β by the smallest functional margin. Both quantities equal 5 in this example, but this is not true in general.

Data: `margin-geometry-4d.csv` from HW08 Q2.

question 3

a.

The assigned skill is `lda-qda-analysis` from BrianChao623's HW7 report, as listed in Issue #4. The full skill is stored in `homework/week-08/SKILL.md` without changes and is included at the end of this report. It was supplied through an explicit request, not installed as an automatic skill.

The agent was asked to use this skill on the SAheart data, with response `chd` coded as 0 and 1. All nine HW7 predictors were used, with `famhist` coded as Absent = 1 and Present = 2. The other predictors were unchanged. No additional preferences were supplied for methods, models, plots, data splits, seeds, or report style.

The analysis finished on October 18, 2026, at 23:05:18 CDT. The model and settings were fixed before evaluating the test set. The run was not stopped early, and no instructions changed the analysis. The separate one-page report is saved as `peer-skill/output/pdf/lda-qda-report.pdf`, and the R code is saved as `peer-skill/run_analysis.R`.

b.

Using seed 43208, the data were split into 370 training and 92 test observations. LDA and QDA were compared using the same five stratified folds within the training set. There were no missing values, no observations were removed, and no model fits failed.

Candidate	CV mistakes	CV error	Failed folds	Selected
LDA	103/370	27.84%	0	Yes
QDA	112/370	30.27%	0	No

LDA had the lower cross-validation error and was selected. It was refitted using all 370 training observations and evaluated once on the reserved test set. There were 25 errors out of 92 observations, giving a test error of 27.17%.

c.

I think the skill is pretty clear and useful. It gives clear steps about comparing LDA and QDA and it keeps the test data separate when choosing the model. The run finished successfully and produce a short report with separate code. However, the instructions and explanations could be more detailed.

1. I think just pointing out the overall error is not enough. The error rate is 27.17%, but the model only got 50% class 1 right, compared with 85% of class0. I think add a small table to show the results for each class would make it easier to see where the model make mistakes.
2. LDA did better than QDA in this run, but the difference in their CV error rates is about 2.43 percentage points. It we spilt the data in different ways, the result may change. Although the report mentioned this, but i think it could explain the uncertainty in a more clear way.
3. The skill asks for a fixed random seed, but theres no defalt one. Different people may choose different seeds and get different data splits. I think providing.a defalt seed will make the experiment easier to repeat

d. GitHub Issue URL

Homework 8 peer review: lda-qda-analysis.

Skill and invocation

The skill is 4333 bytes and is kept outside automatically loaded skill folders. The original invocation used the identical copy in `peer-skill/SKILL.md`:

> Follow the complete local peer-skill/SKILL.md to analyze the SAheart data, with response chd coded as 0 and 1 and all nine HW7 predictors. Code famhist as Absent = 1 and Present = 2, and leave the other predictors unchanged. Let the skill determine the analysis and report.

```
---
name: lda-qda-analysis
description: When explicitly supplied by the user, compare LDA and QDA classifiers using training-only cross-validation
 ↪  and produce a one-page statistical report.
---

# LDA/QDA analysis

## Invocation and inputs

Keep this file outside automatically loaded skill folders. Apply it only when the user explicitly asks to use this
 ↪  file. Obtain the dataset, response, predictor list, coding instructions, and any split constraints from the
 ↪  invocation; do not hard-code a dataset or its results.

## Agent behavior

Use R unless the user requests another language. Execute the analysis and verify numerical results. Preserve the
 ↪  requested predictor coding. Ask for missing information only when it prevents a valid analysis. Explain findings in
 ↪  plain language and avoid claiming a model is best beyond the candidates evaluated.

## Analysis guidelines

1. Inspect class counts, missing values, predictor types, and dimensions. Record coding decisions and report any
 ↪  exclusions. Use a fixed random seed and report it.
2. Reserve a stratified 20% test set before model comparison, unless the user supplies another valid split. Keep test
 ↪  outcomes out of all preprocessing, model selection, and skill revisions. Adjust split and fold counts if class
 ↪  sizes require it, and explain why.
3. Compare ordinary LDA and ordinary QDA using the same numerical predictors and empirical training priors. Optional
 ↪  candidates may use PCA before LDA or QDA, or a weighted average of their posterior probabilities. All classifiers
 ↪  must remain based on LDA, QDA, or their combination. Specify a modest candidate grid before evaluation.
```

```
4. Use the same stratified five-fold cross-validation splits within the training set for every candidate. Fit means,
 ↪  priors, covariances, imputation, scaling, PCA, and any other learned preprocessing using only each fold's training
 ↪  observations; apply those fitted transformations to its validation observations. Select PCA dimensions or
 ↪  probability-blending weights only through this cross-validation.
5. Classify by the largest posterior probability, using a consistent tie rule. Compare candidates by total validation
 ↪  mistakes divided by total validation observations. Report cross-validation misclassification rates. Select the
 ↪  lowest-error valid candidate; prefer the simpler candidate in an exact tie.
6. Check for constant predictors, linear dependence, and singular class covariance matrices. Having more observations
 ↪  than predictors does not guarantee invertibility. Record failed fits and their causes; do not silently substitute a
 ↪  regularized model or omit failed folds from an average. Treat a candidate that fails in any fold as invalid.
7. Make any model or skill revisions using training and cross-validation results only. Freeze the selected
 ↪  configuration, refit it and its preprocessing on all training observations, then evaluate that selected classifier
 ↪  once on the reserved test set. Do not revise the model after seeing test results.

## Required report

Produce a readable analysis report of at most one printed page, containing all of the following:

- Setup: response, predictors and coding, sample and class counts, train/test sizes, seed, and any exclusions.
- Methods: candidate models, covariance assumptions, preprocessing, shared cross-validation folds, and selection rule.
- One compact table: candidate settings, cross-validation misclassification rates, fit failures if present, and the
 ↪  selected model clearly marked.
- One labeled figure: compare candidates' cross-validation misclassification rates; identify the selected candidate and
 ↪  display rates as percentages.
- Final numerical results: selected model, number of predictors or retained components, test mistakes and test sample
 ↪  size, and test misclassification rate.
- Interpretation: explain the main comparison using covariance assumptions and the bias-variance trade-off. Describe
 ↪  limitations, including sample size, model assumptions, and uncertainty from a single test split.

Keep executable code and supplementary output outside the one-page report. Render the report and inspect its page count
 ↪  and readability; shorten content without shrinking text to an unreadable size. Deliver reproducible code
 ↪  separately. Distinguish cross-validation results used for selection from the final test result.
```